Structure of Computer Systems

Android Benchmark Application

Student: Kolumbán Zsolt

Group: 30432

Academic Year: 2023-2024

Table of Contents

1. **Introduction**
    - 1.1 Context
    - 1.2 Objectives
2. **Bibliographic Research**
    - 2.1 What is a Benchmark?
    - 2.2 Benchmarking Methods
    - 2.3 Related Work
    - 2.4 Detailed Analysis of Existing Benchmarks
        - AnTuTu Benchmark
        - Geekbench
3. **Analysis**
    - 3.1 Overview
    - 3.2 Performance Parameters
        - CPU Parameters
        - Memory Parameters
        - GPU Parameters
    - 3.3 Factors Affecting Performance
    - 3.4 Conceptual Benchmarking Methodology
    - 3.5 Benchmark Algorithms
        - 3.5.1 CPU Benchmark Algorithm
        - 3.5.2 Memory Benchmark Algorithm
        - 3.5.3 GPU Benchmark Algorithm
    - 3.7 Expected Output Metrics
4. **Design**
    - 4.1 Overview
    - 4.2 Application Architecture
    - 4.3 UML Use Case Diagram
    - 4.4 UML Class Diagram
    - 4.6 Component Descriptions and Interactions
    - 4.7 Correlation with Theoretical Concepts
5. **Implementation**
    - 5.1 Overview
    - 5.2 Development Environment

5.3 Application Modules
5.3.1 CPU Benchmark Implementation
5.3.2 Memory Benchmark Implementation
5.3.3 GPU Benchmark Implementation
5.4 Multithreading and Execution Control
5.5 Result Computation and Data Handling
5.6 Visualization and User Feedback
5.7 Summary

Bibliography

1. Introduction

1.1 Context

Modern Android smartphones vary widely in performance depending on their hardware (CPU, GPU, and memory). Benchmark applications help evaluate how well a device performs under various workloads, enabling fair comparisons between models and helping users or developers understand system limits.

This project aims to develop a **Benchmark App for Android** that measures CPU, GPU and memory performance using custom test routines. The application will provide a performance score and display results in an intuitive interface.

1.2 Objectives

The main objectives of the project are:

- Develop an Android app capable of running standardized CPU and memory performance tests.
- Display results through clear charts and numerical scores.
- Ensure consistent results across different devices and Android versions.
- Learn and apply efficient multithreading and performance profiling techniques in Android.

2. Bibliographic Research

2.1 What is a Benchmark?

A benchmark is a standardized test designed to evaluate the performance of a system or component under controlled conditions. In smartphones, benchmarks are used

to assess the processing speed, memory performance, and graphical rendering capability of devices.

2.2 Benchmarking Methods

CPU benchmarks usually perform repetitive mathematical or logic operations to test computation speed. Memory benchmarks measure latency and throughput by performing sequential and random data access operations.

Common approaches include:

- **Loop-based computation** (e.g., large number of arithmetic operations)
- **Threaded workloads** to simulate multitasking
- **Data processing tasks** (e.g., JSON parsing or compression)

2.3 Related Work

Existing benchmark applications such as Geekbench or AnTuTu provide complex and proprietary testing algorithms. This project aims to create a simplified, open-source version that demonstrates the key ideas of benchmarking, focusing mainly on CPU and RAM.

2.4 Detailed Analysis of Existing Benchmarks

AnTuTu Benchmark:

AnTuTu is one of the most popular all-in-one benchmarking tools for Android. It evaluates several hardware components, including CPU, GPU, memory, and UX (user experience). The CPU test measures integer and floating-point performance, while the GPU test renders complex 3D scenes to evaluate graphics performance. The memory test measures RAM speed and data transfer rates, and the UX test simulates real-world app scenarios such as data compression, database operations, and interface responsiveness. The final score is a weighted sum of all these individual results.

Advantages:

- Provides a comprehensive overview of overall device performance (CPU, GPU, memory, UX).
- Easy-to-use interface and widely recognized scores for comparison.

- Tests simulate realistic usage scenarios, such as gaming or app launching.
- Large user base provides average scores for most devices, allowing comparison with real-world results.

Disadvantages:

- Proprietary algorithms make it non-transparent, details of weight distribution are not public.
- Some manufacturers have been known to optimize devices specifically for AnTuTu, leading to inflated scores.
- Heavily dependent on software version; scores may vary with app updates.

Geekbench:

Geekbench focuses on CPU and memory performance, using short, cross-platform tests that simulate real-world tasks such as image processing, compression, encryption, and machine learning operations. It provides **single-core** and **multi-core** scores separately to indicate how well a device performs under light and heavy workloads. Geekbench also uses consistent test routines across platforms (Android, iOS, Windows, macOS, Linux) for better comparability.

Advantages:

- Cross-platform design allows fair performance comparisons between different devices and operating systems.
- Separates single-core and multi-core results for detailed insight.
- Transparent about test types and workloads, with publicly available documentation.
- Lightweight and quick to run.

Disadvantages:

- Limited focus—tests mainly CPU and memory, ignoring GPU and UX performance.
- Short test duration can sometimes miss sustained performance (thermal throttling).

3. Analysis

3.1 Overview

The goal of this project is to analyze and evaluate the performance of Android devices by measuring CPU, Memory, and GPU efficiency using synthetic benchmark routines. This chapter describes the theoretical background, parameters, and conceptual methodology used for performance measurement.

3.2 Performance Parameters

The benchmark focuses on three core hardware components:

1. CPU Performance – measures computation speed and multithreaded efficiency.
2. Memory Performance – measures data throughput and latency.
3. GPU Performance – measures graphical rendering and parallel computation power.

CPU Parameters

- Clock Frequency (GHz): Determines how many cycles per second the CPU executes.
- Core Count and Architecture: Defines parallelism and efficiency in multithreaded tasks.
- Instruction per Cycle (IPC): Represents how many instructions can be executed per clock cycle.
- Thermal Throttling: Can reduce frequency under high temperatures, affecting sustained performance.

Memory Parameters

- Bandwidth (MB/s): Rate at which data can be transferred between RAM and CPU.
- Latency (ns): Time delay between requesting and receiving data.

- Cache Hierarchy Efficiency: Determines how well the CPU can reuse recently accessed data.

GPU Parameters

- Fill Rate (MPixels/s): Measures how many pixels the GPU can render per second.
- Shader Performance (GFLOPS): Indicates how fast the GPU can execute floating-point operations.
- Texture Throughput: Measures speed of texture mapping operations.
- Frame Rendering Time (ms/frame): Determines how efficiently the GPU handles 2D/3D workloads.

3.3 Factors Affecting Performance

Performance variations can be caused by:

- **Hardware architecture:** differences in chip design, fabrication process, and core efficiency.
- **Software factors:** background services, operating system version, and app optimizations.
- **Thermal behavior:** overheating can cause frequency throttling.
- **Battery mode:** power-saving modes can cap CPU/GPU performance.
- **Driver and API versions:** GPU performance depends on OpenGL ES / Vulkan driver quality.

3.4 Conceptual Benchmarking Methodology

The conceptual benchmarking process involves three main stages:

1. **Workload Generation:** Define a series of CPU, memory, and GPU tasks that simulate real-world scenarios.
2. **Execution and Measurement:** Run these workloads while measuring time, throughput, and rendering rate.
3. **Normalization and Scoring:** Convert raw data (operations per second, MB/s, FPS) into standardized scores.

3.5 Benchmark Algorithms

3.5.1 CPU Benchmark Algorithm

Goal: Measure computation speed under arithmetic load.

```
start_time ← current_time()
sum ← 0
for i ← 1 to N do
  sum ← sum + (i × i) - (i ÷ 2)
end for
end_time ← current_time()

elapsed_time ← end_time - start_time
CPU_score ← N / elapsed_time
return CPU_score
```

3.5.2 Memory Benchmark Algorithm

Goal: Evaluate RAM throughput and latency.

```
Allocate array A of size array_size

start_time ← current_time()
for i ← 1 to array_size do
        A[i] ← random_value()
end for
for i ← 1 to array_size do
         temp ← A[random_index()]
end for
end_time ← current_time()
elapsed_time ← end_time - start_time
Memory_score ← array_size / elapsed_time
return Memory_score
```

3.5.3 GPU Benchmark Algorithm

Goal: Measure GPU performance in rendering and parallel computations.

Concept: The GPU benchmark renders a series of 3D objects or complex graphical scenes repeatedly to measure rendering speed and computational power.

```
Initialize 3D scene with multiple textured objects
start_time ← current_time()
for frame ← 1 to num_frames do
        RenderScene()
        ApplyLighting()
        ApplyShaders()
end for
end_time ← current_time()
elapsed_time ← end_time - start_time
frames_per_second ← num_frames / elapsed_time
GPU_score ← frames_per_second × complexity_factor
return GPU_score
```

3.7 Expected Output Metrics

Metric	Description	Unit
CPU Score	Computation speed (iterations per second)	arbitrary units
Memory Score	Memory throughput (data processed per second)	MB/s
GPU Score	Rendering throughput (frames per second × complexity factor)	FPS / points
Total Benchmark Score	Weighted average of all three components	points

4. Design

4.1 Overview

This chapter presents the design and structure of the Android Benchmark Application. It explains how the theoretical concepts described in Chapter 3 (CPU, Memory, and GPU performance measurement) are applied in practice through the app's architecture, modules, and component interactions.

The application is divided into several software layers to ensure modularity, scalability, and ease of maintenance.

4.2 Application Architecture

The project follows a **modular architecture**, separating **user interface**, **benchmark logic**, and **data management**.

Architecture Layers:

1. **User Interface (UI) Layer**
    - Provides a graphical interface for the user to start benchmarks, view results, and compare scores.
    - Implements intuitive charts and progress indicators.
2. **Benchmark Engine (Core Logic)**
    - Contains the algorithms for CPU, Memory, and GPU performance tests.
    - Manages task scheduling, timing, and scoring normalization.
3. **Data Management Layer**
    - Responsible for storing benchmark results (locally or in shared preferences/database).
    - Handles normalization and scaling of scores.
4. **Visualization Module**
    - Converts raw performance data into readable charts and graphical representations.
    - Displays CPU, Memory, and GPU scores side by side for easy comparison.

4.3 UML Use Case Diagram

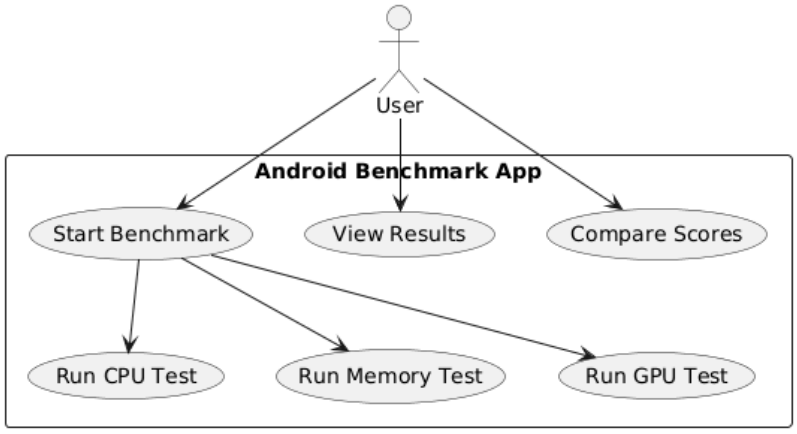

Figure 4.2. Use Case Diagram

(Figure 4.2 shows the main use cases of the application: the user starts the benchmark, views results, and compares scores. The system executes benchmark routines and generates performance scores.)

Actors and Use Cases:

- **User**: Starts benchmark, views results, and compares with previous runs.
- **System**: Executes benchmarks, collects raw data, and calculates scores.

4.4 UML Class Diagram

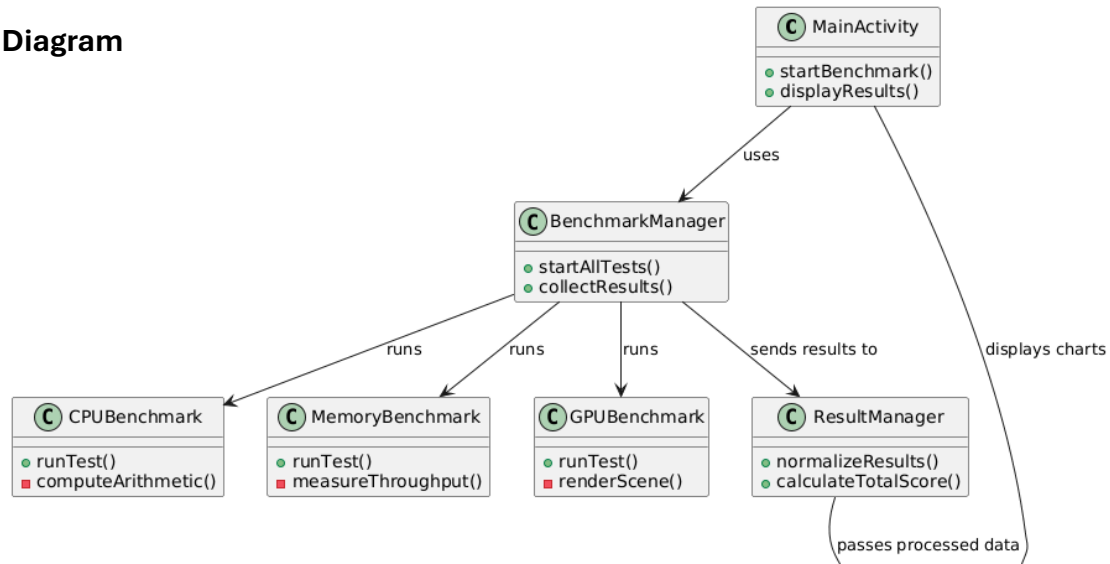

Figure 4.3. Class Diagram of Benchmark Application

(Figure 4.3 presents the main classes and their relationships.)

4.6 Component Descriptions and Interactions

Component	Role	Interaction
UI Layer	Displays buttons, charts, and results.	Interacts with BenchmarkManager to start tests.
BenchmarkManager	Core controller coordinating benchmarks.	Calls CPU, Memory, and GPU modules; collects results.
CPUBenchmark	Executes mathematical workloads.	Returns computation speed to manager.
MemoryBenchmark	Executes memory allocation and access tests.	Returns throughput and latency results.
GPUBenchmark	Runs OpenGL/Vulkan workloads.	Returns frame rate and rendering performance.
ResultManager	Processes and normalizes scores.	Sends data to Visualization module.
Visualization (ChartView)	Displays final scores and comparisons.	Receives data from ResultManager.

4.7 Correlation with Theoretical Concepts

Each module implements the theoretical principles described in Chapter 3:

- **CPU Benchmark:** Implements arithmetic operation loops to test computation speed (Section 3.5.1).

- **Memory Benchmark:** Uses read/write tests to evaluate throughput and latency (Section 3.5.2).
- **GPU Benchmark:** Renders 3D scenes to assess graphical computation power (Section 3.5.3).
- **Scoring System:** Implements normalization and aggregation (Section 3.4).

Thus, the design directly translates theoretical performance parameters into measurable, software-driven experiments.

5. Implementation

5.1 Overview

This chapter describes the implementation of the Android Benchmark Application, detailing how each component from the design phase was developed and integrated. The app was implemented using **Android Studio** with **Kotlin** as the primary language, utilizing **OpenGL ES** for GPU testing and standard Android libraries for UI and data handling. The implementation strictly follows the modular architecture described in Chapter 4, ensuring separation between the user interface, benchmarking logic, and result visualization.

5.2 Development Environment

Tool / Framework	Description
Android Studio	Integrated Development Environment used for app development and debugging.
Kotlin	Primary programming language for Android components.
OpenGL ES 2.0	Used for GPU rendering tests.
MPAndroidChart	Library used to visualize benchmark results as charts.
Coroutines / Threads	Used to handle asynchronous execution of benchmark tasks.

5.3 Application Modules

5.3.1 CPU Benchmark Implementation

The **CPU Benchmark** measures raw computation performance using a **bubble sort** algorithm on a randomly generated integer array. The algorithm's total execution time is used to derive a performance score inversely proportional to the duration.

5.3.2 Memory Benchmark Implementation

The **Memory Benchmark** simulates frequent read and write operations on an integer array to measure memory throughput and latency. The duration of these operations determines the memory score.

5.3.3 GPU Benchmark Implementation

The **GPU Benchmark** evaluates rendering performance using **OpenGL ES 2.0** by rendering multiple 3D cube objects in an offscreen buffer. The system measures frames per second (FPS) to compute the GPU score.

5.4 Multithreading and Execution Control

Each benchmark (CPU, Memory, GPU) runs on a background thread to prevent blocking the Android main UI thread. This is achieved using Kotlin coroutines or background Thread execution within the benchmark manager.

5.5 Result Computation and Data Handling

The benchmark manager aggregates raw durations from all tests and computes normalized scores for comparison.
Results are then stored using **SharedPreferences** for later retrieval and visual comparison.

5.6 Visualization and User Feedback

Results are visualized in the app using progress indicators and charts. The UI provides a comparison view between CPU, Memory, and GPU scores using a simple bar chart or progress bar. Users can view:

- Individual benchmark scores.
- Total performance index.

5.7 Summary

This implementation successfully integrates three independent benchmarking modules — CPU, Memory, and GPU — each utilizing realistic workloads and precise timing mechanisms.
The modular structure allows easy expansion (e.g., adding disk or UX benchmarks).

By combining native Kotlin performance and OpenGL ES rendering, the system provides a balanced and transparent measurement of Android device performance.

Bibliography

1. Geekbench Official Website. https://www.geekbench.com
2. AnTuTu Benchmark. https://www.antutu.com
3. https://stackoverflow.com/questions/48364483/how-to-test-the-cpu-gpu-ram-utilization-of-the-given-android-app-accurately